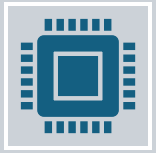
AI, GenAI & Agentic AI for Automotive Leaders and Developers

Surendra Panpaliya

Prerequisites

Basic knowledge of Python (functions, control flow)

Familiarity with automotive workflows,



systems, and product lifecycles

Lab Setup Requirements

Laptop with Python 3.10+, Jupyter Notebook, VS Code

Libraries: numpy, pandas, scikit-learn, matplotlib, openai, langchain, chromadb

Access to Azure OpenAI or Google Gemini APIs

Internet connectivity, Excel, and sample automotive datasets

🎓 Trainer Deliverables

- Pre-reading PDFs on AI, GenAI, Prompting
- Setup guide for labs (for technical team)
- Sample notebooks and visual examples
- Hackathon guidance
- Final review and open Q&A

Program Outcomes for Management Teams

Clearly understand AI, GenAI, and Agentic AI in business context

Evaluate where to invest in AI inside your automotive enterprise

Identify use cases with measurable ROI

Build leadership confidence in leveraging AI responsibly

Align technical and business teams on AI vision

Day-Wise Outline

Day 1: AI/ML/DL – Foundation for Automotive Innovation

Day 2: AI Leadership, Risk Management & Storytelling with Data

Day 3: Generative AI and Document Automation

Day 4: Strategic Thinking – Competing in the AI Age

Day 5: Introduction to Agentic AI and Innovation Hackathon

Day 5

Introduction to Agentic AI and

Innovation Hackathon

Objective: Introduce leaders to AI agents and

how they automate decision-making and tasks.

Day 5

What is Agentic AI (Simple Explanation)

Agents, Planners, Executors

Business Role View

Popular Tools: AutoGen, LangGraph

(only conceptual overview)

Day 5

Use Case: AI Assistant for Vehicle Configurations

Demo: Ask an AI agent about car specs

and receive tailored suggestions

Mini Hackathon: Team showcase of AI ideas

(no coding required for leaders)

Real-Time Use Case

AI Agent That Reads Spec Sheet and Answers Questions

Use Case Overview

Objective: Build an AI agent that can:

Read a **technical spec sheet**

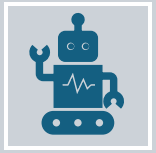
Answer relevant **technical and process questions**

Use tools like **web search, calculator, code interpreter**

Business Need

Technical documents (e.g., API specs, system architecture, release notes) are long and complex.

Engineers & Managers need quick, accurate answers without scanning dozens of pages.



AI agent can act like a **24x7 Technical Assistant**.

Real Use Case in Software Deployment

Scenario:

You are deploying a payment microservice.

You have a 25-page API spec.

Questions arise

- What is the timeout for /pay/process endpoint?
- Which service calls are asynchronous?
- What is the max retry count?
- Instead of scanning the document,
- the agent answers instantly.

Step-by-Step Execution

Surendra Panpaliya

Agent Setup

Use LangChain or AutoGen

Assign role: “Spec QA Assistant”

Document Loading

Upload the spec sheet

(PDF/Markdown/JSON)

Use LangChain’s DocumentLoader

(e.g., PyPDFLoader)

Text Splitting & Indexing

Split into chunks

using RecursiveTextSplitter

Create vector store

using FAISS or Chroma

Add Tools

Search tool: To check recent documentation updates

Calculator tool: For latency, retry, SLA questions

Python REPL: For running small logic/code blocks

Query Handling

User types question:

“What’s the retry logic for payment API?”

Agent retrieves chunk from

vector DB + calculates SLA

Answer + Reasoning Output

Returns answer with

page ref,

explanation,

or logs

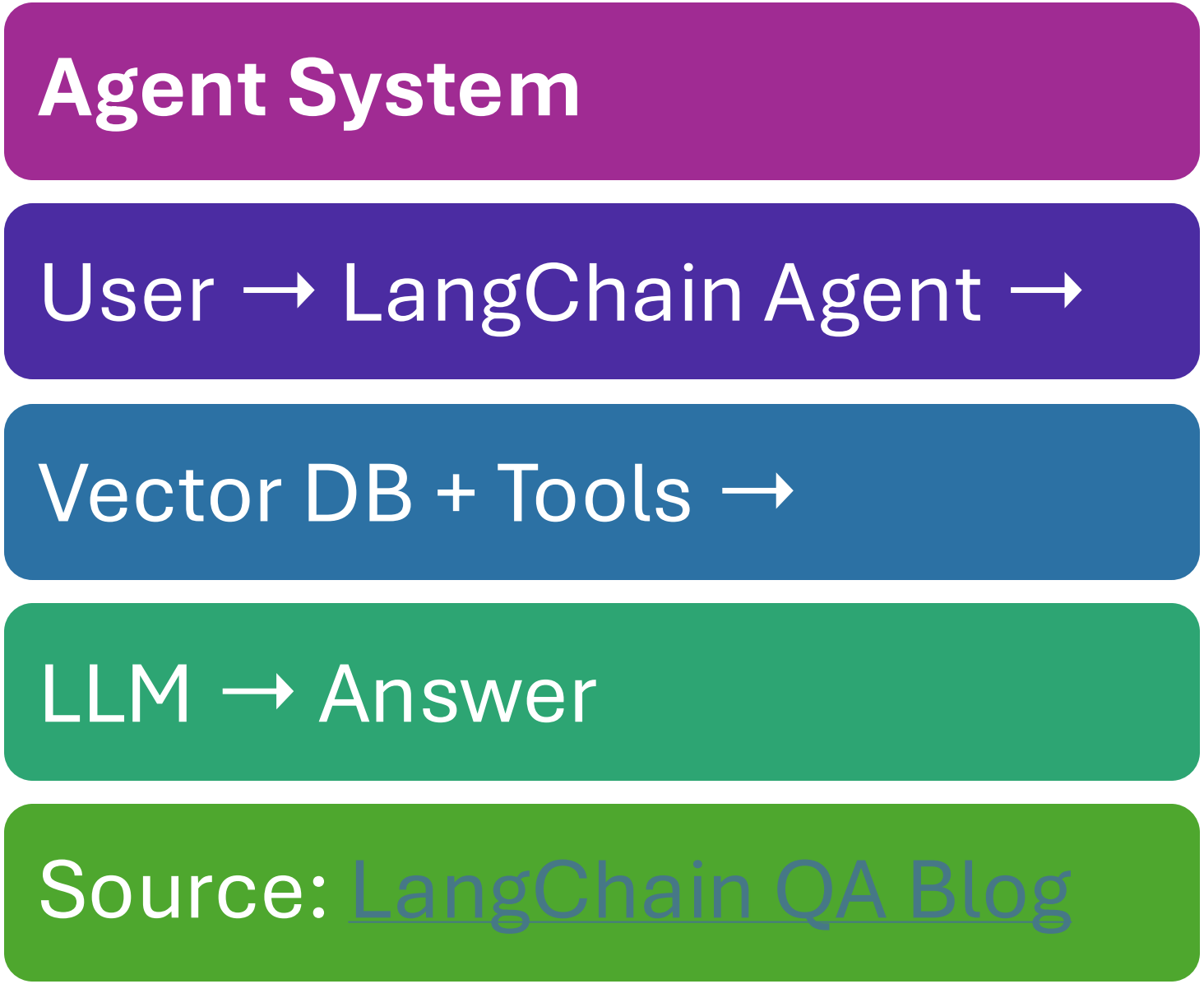
Tools & Libraries Used

Tool / Library	Purpose
LangChain	Agent framework
FAISS / Chroma	Embedding + fast vector retrieval
OpenAI / Azure GPT	LLM for understanding questions
PyPDFLoader	Load PDF spec sheets
Python REPL Tool	Execute logic, math, transforms
SerpAPI / Tavily	Add web search capability

Benefits by Role

Role	Benefit
PM	Get real-time insights from architecture docs
Delivery Manager	Validate service behavior before deployment
Infra Team	Query capacity planning, retry rules, limits

Architecture Diagram



Reference Resources

LangChain QA with PDF:

https://blog.langchain.dev/qa-over-pdfs/

AutoGen Tool Usage:

https://github.com/microsoft/autogen

Tools for LangChain Agents:
https://docs.langchain.com/docs/components/tools/

Reference Resources

Vector DB Chroma:

https://docs.trychroma.com/

FAISS by Meta:

https://github.com/facebookresearch/faiss

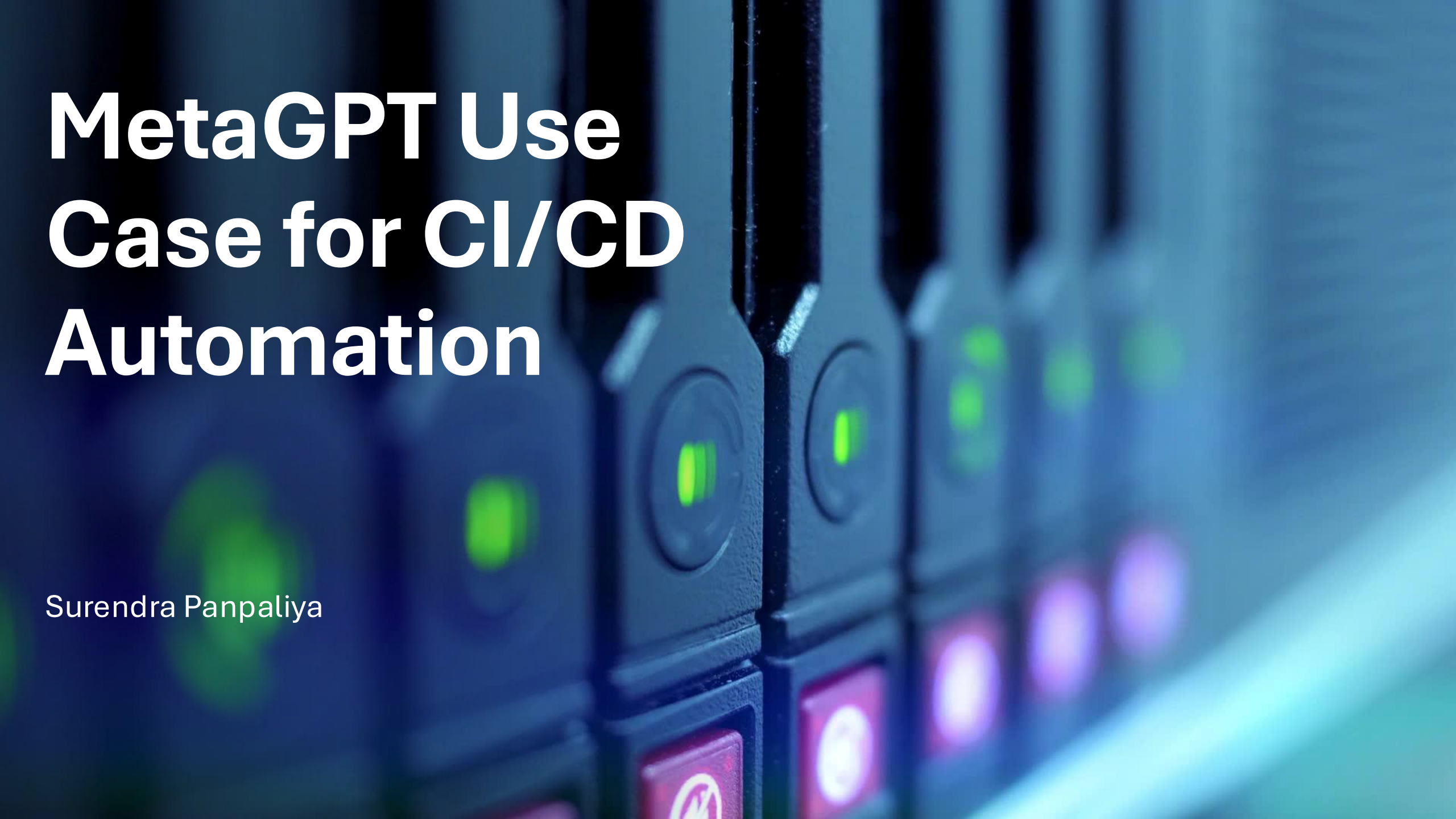
Summary

Building an AI Agent that understands spec sheets:

Reduces manual lookup time

Improves clarity & consistency in team discussions

Helps avoid deployment delays due to confusion or gaps

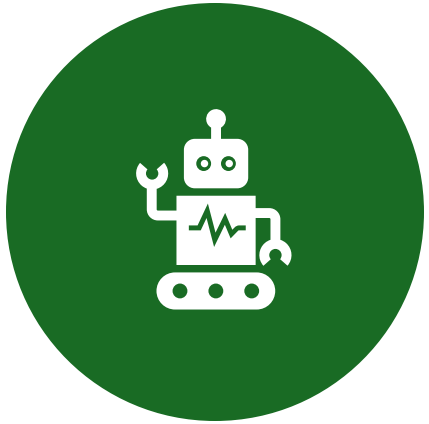


MetaGPT Use Case for CI/CD Automation

Surendra Panpaliya

What is MetaGPT?

OPEN-SOURCE AGENTIC AI FRAMEWORK

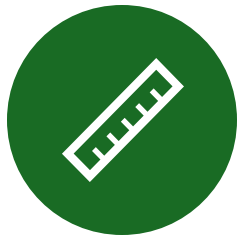


SIMULATES A **MULTI-ROLE SOFTWARE TEAM**

USING LLM-POWERED AGENTS

Roles include

PRODUCT MANAGER



ARCHITECT,

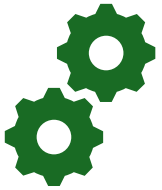
ENGINEER,

QA,

DEVOPS

Why It Matters?

Automates software planning, coding,



testing, and CI/CD workflows

Reduces manual effort and

speeds up delivery cycles

Problem Statement – CI/CD Challenges

CI/CD Professionals Face:

Writing automation logic under pressure

Managing quality documentation & audits

Debugging/refactoring pipelines

Incident responses with traceability

Solution – MetaGPT for CI/CD

Use MetaGPT agents to automate:

Software design

Code generation

Testing

CI/CD pipeline setup

Deployment

MetaGPT Agent Roles

Agent Role	Responsibility
Product Manager	Defines requirements and user stories
Architect	Designs architecture, dependencies
Engineer	Writes app code, Dockerfile, task definitions
QA Agent	Generates tests, coverage, health checks
DevOps Agent	Creates CI/CD pipeline and rollback strategy

Use Case Objective

Build an automated CI/CD pipeline for a Flask API using:

- GitHub Actions
- Unit tests with pytest
- Docker containers
- Deployment to AWS ECS
- Rollback on failure

Input Prompt to MetaGPT

Prompt:

“Build a CI/CD pipeline for a Flask API

using GitHub Actions and deploy it to AWS ECS.

Include unit tests and rollback support.”

Input Prompt to MetaGPT

MetaGPT agents respond by

collaborating to deliver outputs.

MetaGPT Agent Collaboration Flow

PM Agent → Creates structured task

Architect Agent → Designs ECS system,

Docker, CloudWatch

Engineer Agent → Builds code,

Dockerfile, app

MetaGPT Agent Collaboration Flow

QA Agent → Writes tests and

validation logic

DevOps Agent → Creates ci-cd.yml

with rollback logic

Generated Artifacts

- ci-cd.yml:
- GitHub Actions workflow
- Dockerfile:
- Optimized for Flask + pip freeze
- pytest scripts:
- Unit testing coverage

Generated Artifacts

README.md:

Auto-documented CI/CD flow

cloudwatch_alerts.json:

Optional monitoring setup

Architecture Diagram

User Prompt

↓

MetaGPT Task Parser (PM Agent)

↓

Role Agents (Engineer, DevOps, QA, Architect)

↓

Code + CI/CD YAML + Test + Docs → Git Repo

Benefits for CI/CD Teams

Feature	Impact
Multi-agent collaboration	Reduces scripting workload
Task traceability	Maps each artifact to agent origin
Modular output	Files are separated and production-ready
DevSecOps Alignment	Can integrate Vault, secrets, security

Best Practices for Using MetaGPT

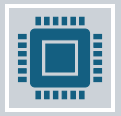
Write structured prompts with clear outcomes

Use sandbox/testing for security-sensitive workflows

Include human-in-the-loop validation

Best Practices for Using MetaGPT

Customize agents for platform-specific needs



EKS, GitLab, Azure DevOps

Commit generated files to Git

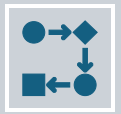
with review policies

Limitations & Recommendations

- Not recommended for highly confidential workflows
- Review IAM keys, access policies manually
- Use audit trails/logs to ensure traceability

Conclusion

MetaGPT = AI-Powered Multi-Agent CI/CD Automation



Saves time in planning, scripting, deploying

Bridges roles (PM, Dev, QA, DevOps)

with agent collaboration

Conclusion

Perfect for teams

focused on **scalability,**

speed, and

DevSecOps maturity

📺 Demo Video

MetaGPT Setup:

- Launch a Startup with One ✍️ Prompt!
- https://www.youtube.com/watch?v=nqZlTV_L6Ao

- MetaGPT: a Multi-Agent Framework to Automate Your Software Company
- https://youtu.be/ccx850jrk9U

Resources

🔗 **MetaGPT GitHub:**

https://github.com/geekan/MetaGPT

🔗 **MetaGPT-Pilot:**

https://github.com/bensadeghi/metagpt-pilot

📰 **Intro Article:** MetaGPT: Multi-Agent Coding Framework

https://medium.com/p/4b6ae747cc36

Try it

https://github.com/geekan/MetaGPT

GitLab Duo

GitLab Duo + Agentic AI

How GitLab Embraces Agentic Design?

What is GitLab Duo?

AI-powered **DevSecOps suite**

embedded within GitLab

Integrated across the entire SDLC:

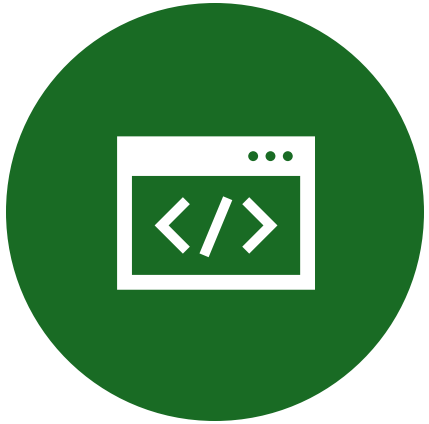
planning → coding → testing → deploying

Provides AI assistance for

- Code suggestions
- Security audits
- Test generation
- CI/CD pipeline optimization
- Natural language to YAML/code generation

Agentic AI – Quick Recap

AGENTIC AI = SYSTEMS COMPOSED OF



MODULAR, INTERACTIVE

AI AGENTS WITH SPECIFIC ROLES

Agentic AI – Quick Recap

Characteristics:

Role-based responsibilities (PM, QA, DevOps)

Goal-driven, autonomous or semi-autonomous workflows

Multi-tool, multi-agent orchestration

Is GitLab Duo an Agentic AI Framework?

✅ Yes – Functionally aligned,

although **not explicitly branded** as such

Is GitLab Duo an Agentic AI Framework?

Agentic Pattern	GitLab Duo Feature
Planning Agent	Auto-generates issues, epics
Coding Agent	Code suggestions in Web IDE
Security Agent	Auto-fixes IaC, container issues
Release Agent	Optimizes .gitlab-ci.yml
Assistant Agent	Duo Chat for repo/pipeline Q&A

GitLab Duo Architecture (Implicit Agentic View)

User Request

↓

Duo Chat Interface (LLM)

↓

[Role-based Submodules]

GitLab Duo Architecture (Implicit Agentic View)

[Role-based Submodules]

→ Code Suggestion Agent

→ Test Generation Agent

→ Security Scan Agent

→ Pipeline Review Agent

↓

Web IDE + CI/CD Insights

GitLab Duo vs MetaGPT vs LangGraph

Feature / Framework	GitLab Duo	MetaGPT	LangGraph
Integration Layer	GitLab Native	External (GitHub, CLI)	Custom orchestration
Agent Visibility	Implicit Roles	Explicit (PM, QA, DevOps)	Graph-based nodes
Multi-Agent Conversation	No	Yes	Yes (LangGraph flow)
CI/CD Pipeline Automation	✅ Built-in	✅ Code generated	🟡 External logic
Tool Extensibility	🟡 Limited to GitLab	✅ Custom Python tools	✅ Full modularity

Strategic Benefits by Role

Role	GitLab Duo Benefit
CI/CD Engineers	Auto-optimize jobs, detect bottlenecks
Project Managers	Convert user stories to structured issues/epics
Delivery Managers	Monitor pipeline health, risk analysis
SRE / Platform Teams	Reduce incident mean-time-to-resolution (MTTR)

Summary

GitLab Duo **mimics Agentic AI behavior**

within a DevSecOps platform

While MetaGPT and LangGraph

offer **explicit agent orchestration**

Summary

Duo delivers:

Embedded intelligence

Context-aware suggestions

Enterprise-ready DevOps integration

Summary

Together, these tools highlight

how Agentic AI is evolving

across platforms

Resources

🔗 GitLab Duo Overview:

https://about.gitlab.com/gitlab-duo/

📄 GitLab AI Vision:

https://about.gitlab.com/blog/2023/06/29/gitlab-ai-devsecops-roadmap/

Resources

📺 Duo Demo:
https://www.youtube.com/watch?v=ddKiwhdAPY8

🧠 MetaGPT Repo: https://github.com/geekan/MetaGPT

🧠 LangGraph Docs: https://docs.langchain.com/langgraph

Final Thought

- GitLab Duo represents
- the **enterprise-friendly implementation**
- of Agentic AI for CI/CD.
- Use Duo for **native GitLab optimization**

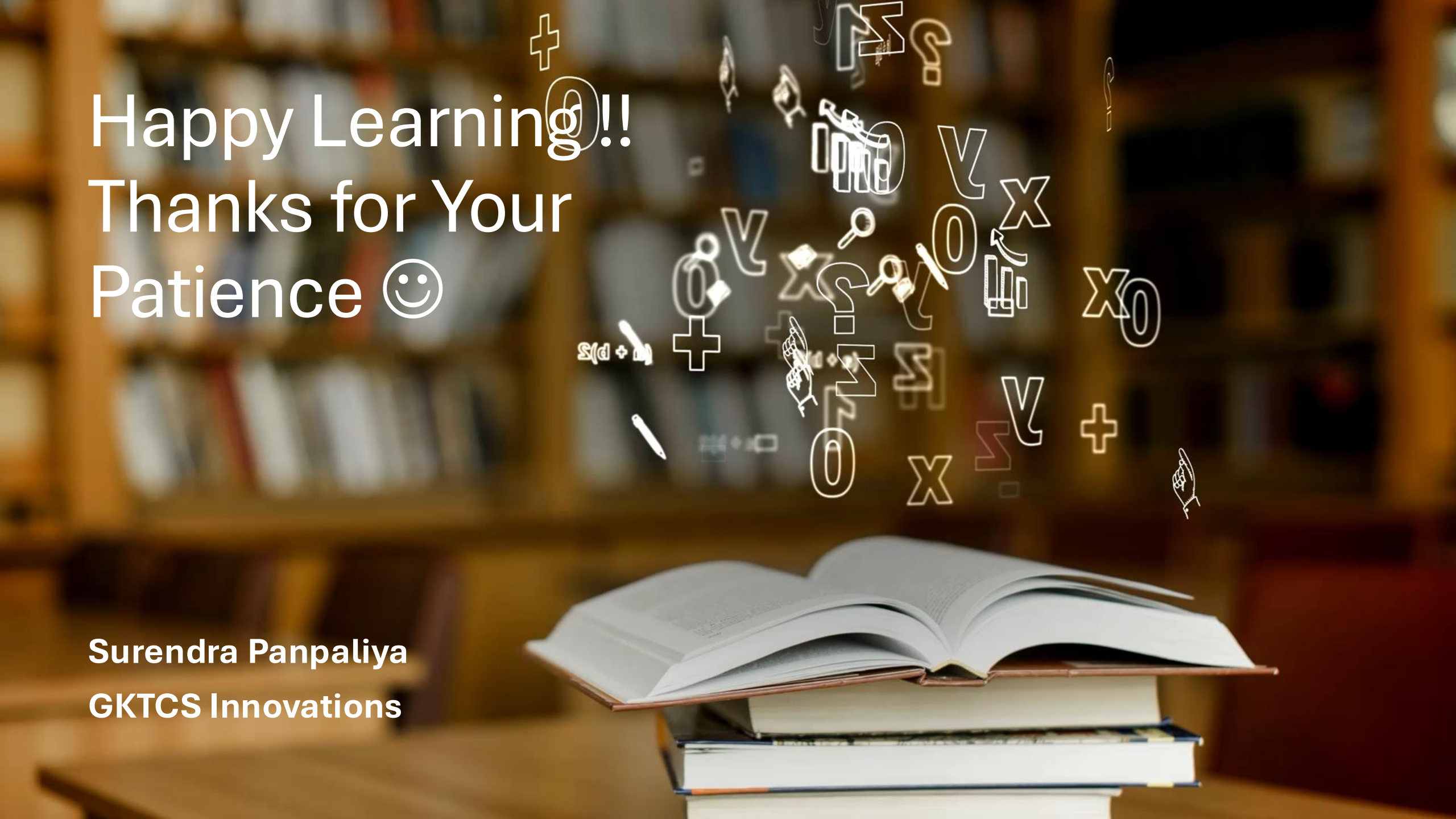
Final Thought

Use MetaGPT or LangGraph

for **custom agent workflows**

Modern DevOps is moving from

pipelines to agent-driven platforms.



Surendra Panpaliya

GKTCS Innovations